Chapter 7

Kleene's Theorem

7.1 Kleene's Theorem

The following theorem is the most important and fundamental result in the theory of FA's:

Theorem 6 *Any language that can be defined by either*

- *regular expression, or*
- *finite automata, or*
- *transition graph*

can be defined by all three methods.

Proof. The proof has three parts:

Part 1: (FA $\Rightarrow$ TG) Every language that can be defined by an *FA* can also be defined by a *transition graph*.

Part 2: (TG $\Rightarrow$ RegExp) Every language that can be defined by a *transition graph* can also be defined by a *regular expression*.

Part 3: (RegExp $\Rightarrow$ FA) Every language that can be defined by a *regular expression* can also be defined by an *FA*.

7.2 Proof of Part 1: FA $\Rightarrow$ TG

- We previously saw that every FA is also a transition graph.
- Hence, any language that has been defined by a FA can also be defined by a transition graph.

7.3 Proof of Part 2: TG $\Rightarrow$ RegExp

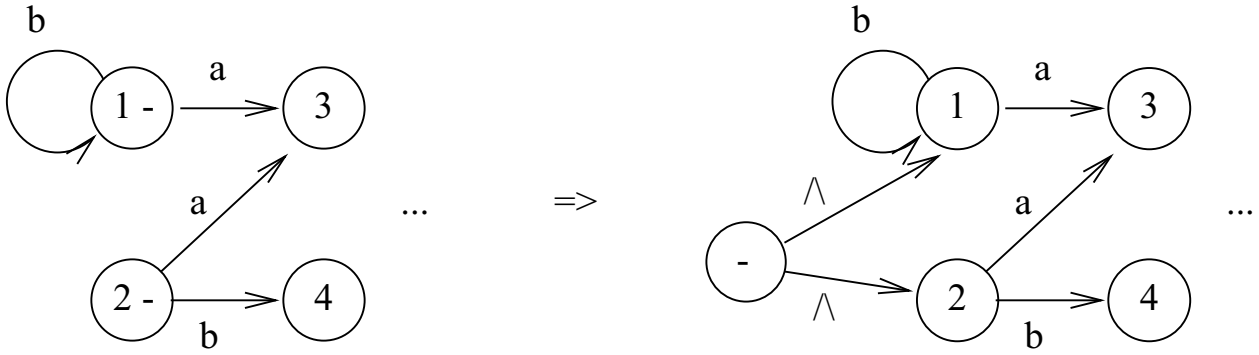
- We will give a constructive algorithm for proving part 2.
- Thus, we will describe an algorithm to take any transition graph T and form a regular expression corresponding to it.
- The algorithm will work for any transition graph T.
- The algorithm will finish in finite time.

An overview of the algorithm is as follows:

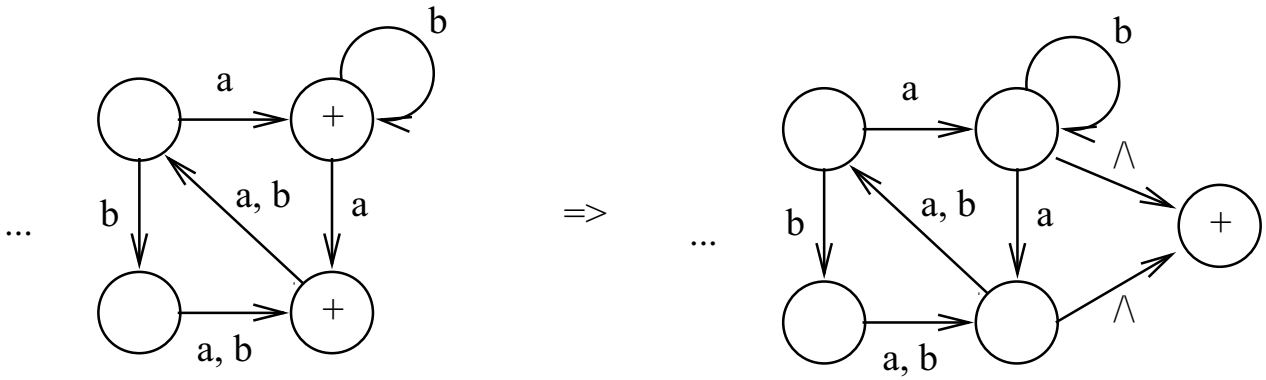
- Start with any transition graph T.
- First, transform it into an equivalent transition graph having only one start state and one final state.
- In each following step, eliminate either some states or some arcs by transforming the TG into another equivalent one.
- We do this by replacing the strings labelling arcs with regular expressions.
- We can traverse an arc labelled with a regular expression using any string that can be generated by the regular expression.
- End up with a TG having only two states, start and final, and one arc going from start to final.
- The final TG will have a regular expression on its one arc
- Note that in each step we eliminate some states or arcs.
- Since the original TG has a finite number of states and arcs, the algorithm will terminate in a finite number of iterations.

Algorithm:

1. If T has more than one start state, add a new state and add arcs labeled Λ going to each of the original start states.

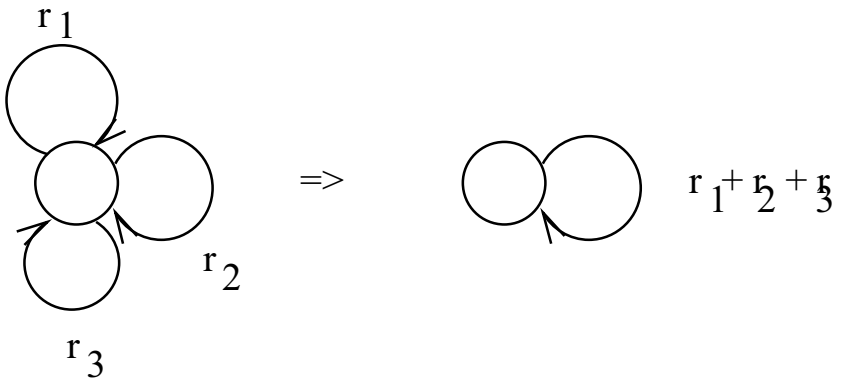


2. If T has more than one final state, add a new state and add arcs labeled Λ going from each of the original final states to the new state. Need to make sure the final state is different than the start state.

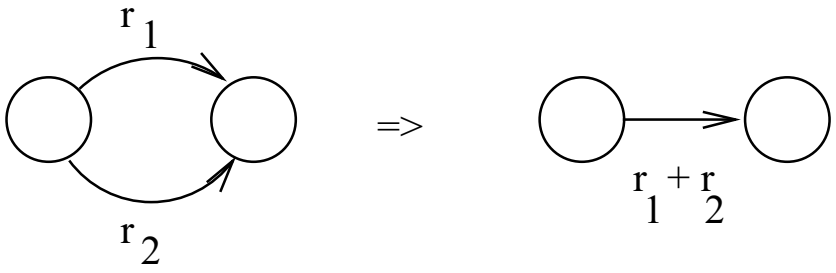


3. Now we give an iterative procedure for eliminating states and arcs

   (a) If T has some state with $n > 1$ loops circling back to itself, where the loops are labeled with regular expressions $\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_n$, then replace the n loops with a single loop labeled with the regular expression $\mathbf{r}_1 + \mathbf{r}_2 + \cdots + \mathbf{r}_n$.



   (b) If two states are connected by $n > 1$ direct arcs in the same direction, where the arcs are labelled with the regular expressions $\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_n$, then replace the n arcs with a single arc labeled with the regular expression $\mathbf{r}_1 + \mathbf{r}_2 + \cdots + \mathbf{r}_n$.

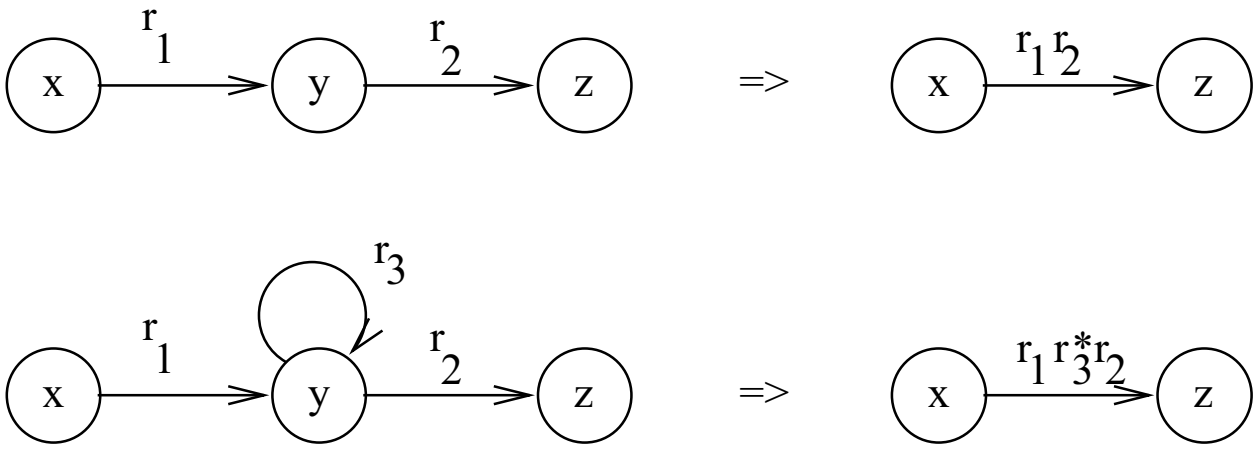


(c) *Bypass operation*:

i. If there are three states x, y, z such that

- there is an arc from x to y labelled with the regular expression $\mathbf{r}_1$ and
- an arc from y to z labelled with the regular expression $\mathbf{r}_2$,

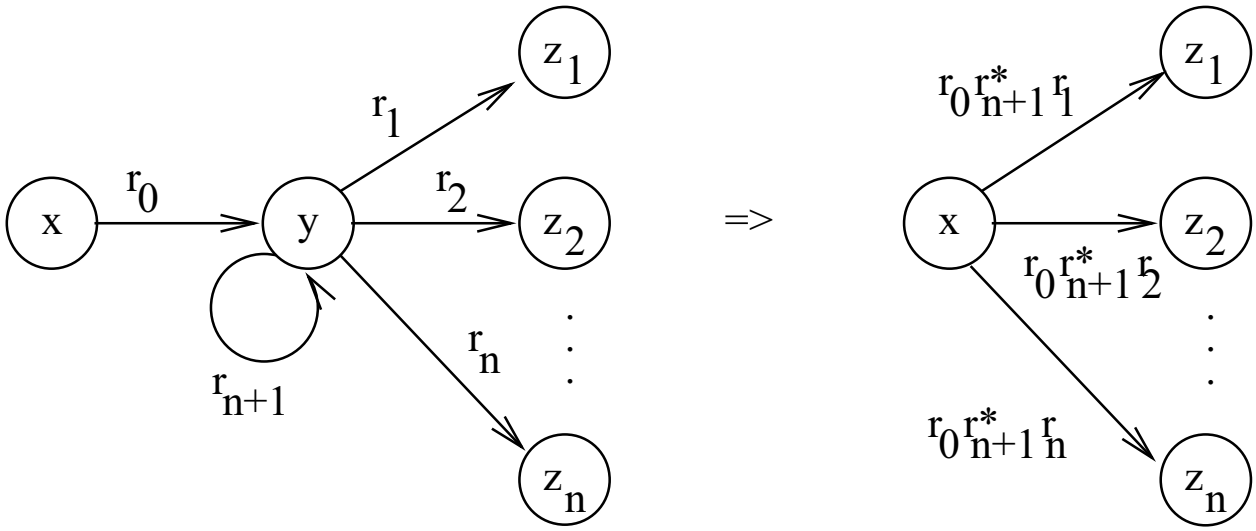
then replace the two arcs and the state y with a single arc from x to z labelled with the regular expression $\mathbf{r}_1\mathbf{r}_2$.



ii. If there are

- $n+2$ states $x, y, z_1, z_2, \ldots, z_n$ such that there is an arc from x to y labelled with the regular expression $\mathbf{r}_0$, and
- an arc from y to z_i, $i = 1, 2, \ldots, n$, labelled with the regular expression $\mathbf{r}_i$, and
- an arc from y back to itself labelled with regular expression $\mathbf{r}_{n+1}$,

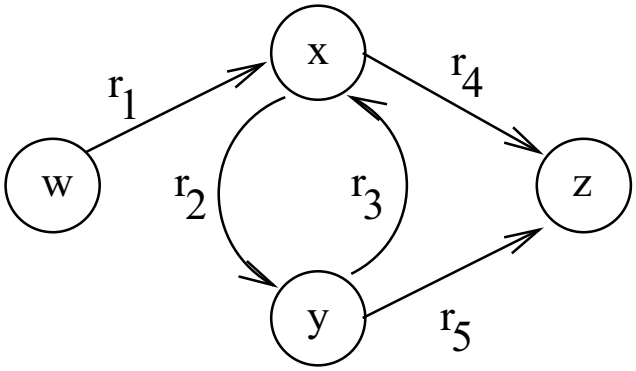
then replace the $n + 1$ original arcs and the state y with n arcs from x to z_i, $i = 1, 2, \ldots, n$, each labelled with the regular expression $\mathbf{r}_0\mathbf{r}_{n+1}\mathbf{r}_i$.

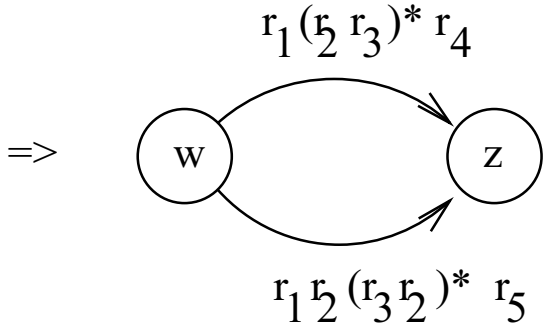


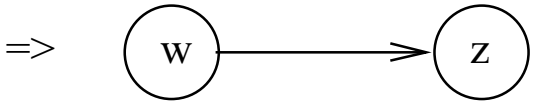
iii. If any other arcs led directly to y, divert them directly to the z_i's.

iv. Need to make sure that all paths possible in the original TG are still possible after the bypass operation.

- Example

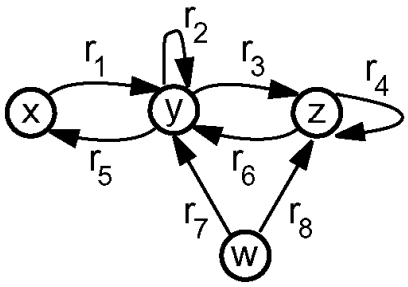




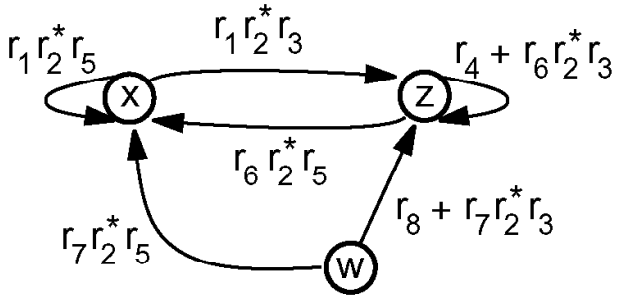


$$r_1 (r_2 r_3)^* r_4 \;+\; r_1 r_2 (r_3 r_2)^* r_5$$

- Example:

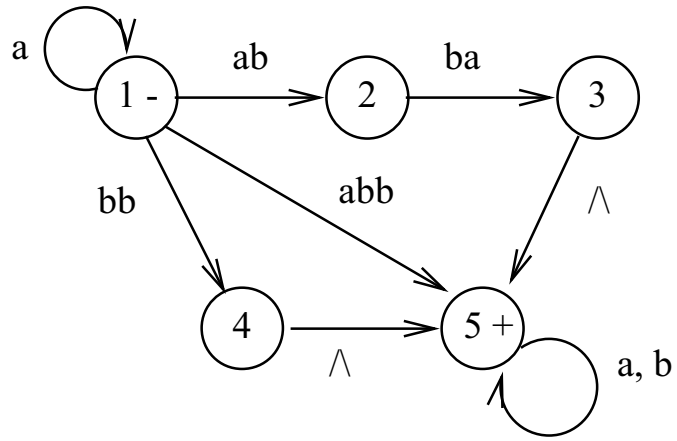


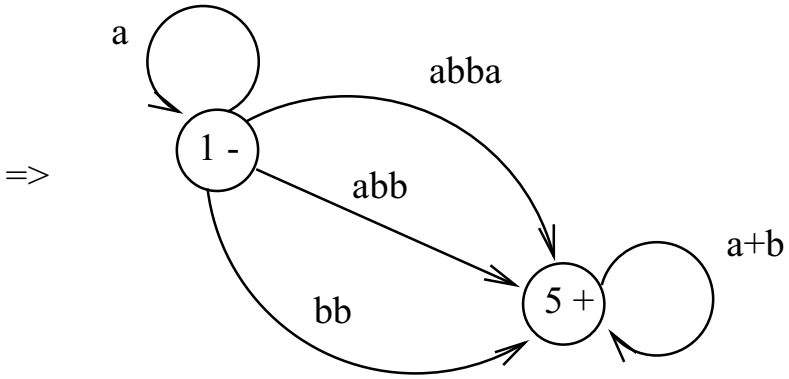
  - Suppose we want to get rid of state y.
  - Need to account for all paths that go through state y.
  - There are arcs coming from x, w, and z going into y.
  - There are arcs from y to x and z.
  - Thus, we need to account for each possible path from a state having an arc into y (i.e., x, w, z) to each state having an arc from y (i.e., x, z)
  - Thus, we need to account for the paths from
    - x to y to x, which has regular expression $\mathbf{r}_1\mathbf{r}_2^*\mathbf{r}_5$
    - x to y to z, which has regular expression $\mathbf{r}_1\mathbf{r}_2^*\mathbf{r}_3$
    - w to y to x, which has regular expression $\mathbf{r}_7\mathbf{r}_2^*\mathbf{r}_5$
    - w to y to z, which has regular expression $\mathbf{r}_7\mathbf{r}_2^*\mathbf{r}_3$
    - z to y to x, which has regular expression $\mathbf{r}_6\mathbf{r}_2^*\mathbf{r}_5$
    - z to y to z, which has regular expression $\mathbf{r}_6\mathbf{r}_2^*\mathbf{r}_3$
  - Thus, after eliminating state y, we get the following:

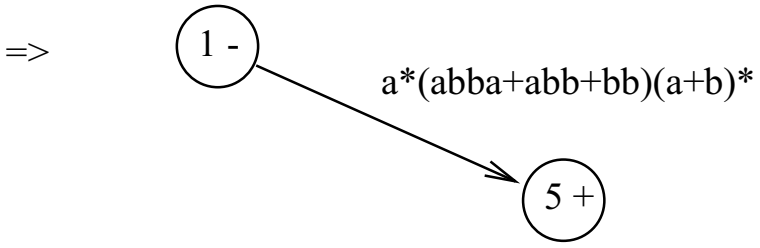


v. Never delete the unique start or final state.

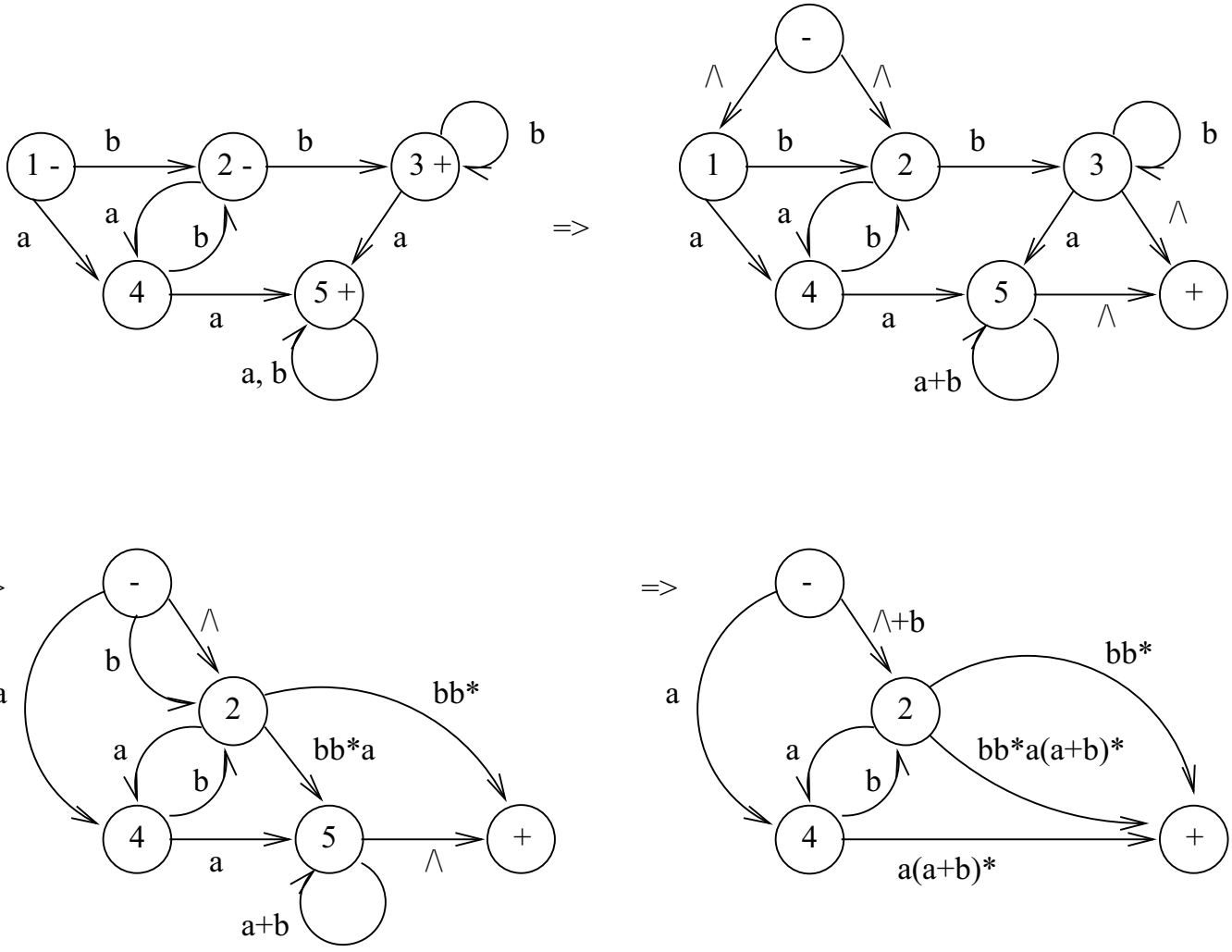
Example:

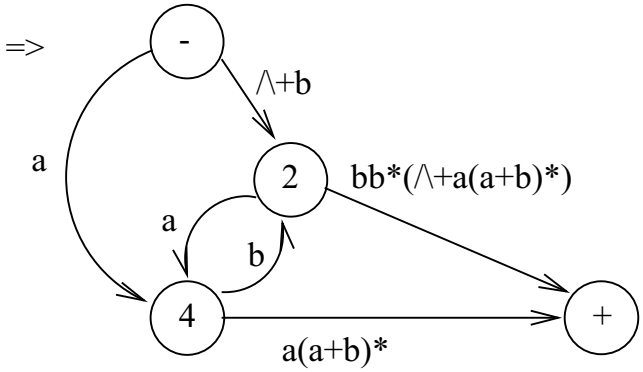


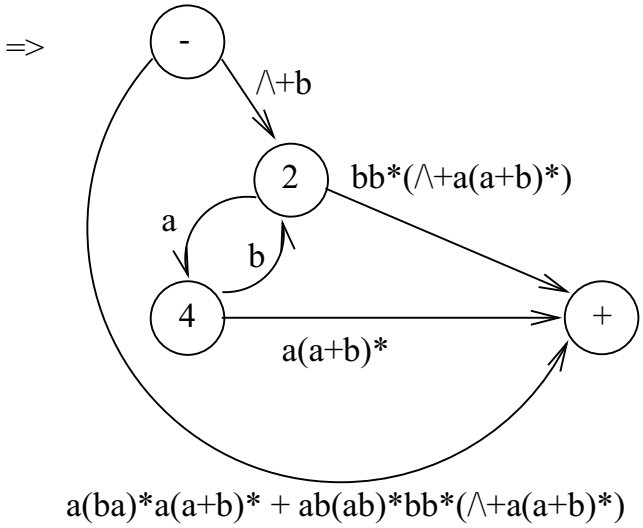


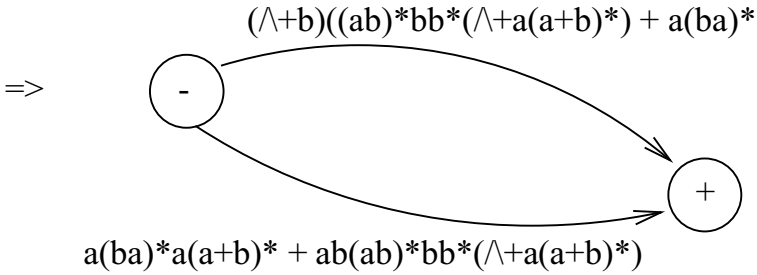


Example:









=> (/\+b)((ab)*bb*(/\+a(a+b)*) + a(ba)*a(a+b)*) + a(ba)*a(a+b)* + ab(ab)*bb*(/\+a(a+b)*)